

Function Convexity Interpolation Notes

James Evans

07/23/2025

Function Convexity Interpolation

Tony Deng

1 Mathematical Background for convexity

We are trying to solve the following question: Given two smooth convex functions $f, g : A \rightarrow \mathbb{R}$ where $A \subset \mathbb{R}^d$ and their gradient, we want to find the minimum β such that $f + \beta g$ is still convex. Equivalently, we want to find the minimum β such that for all $x, z \in A$,

$$(f + \beta g)(z) - (f + \beta g)(x) \geq \nabla(f + \beta g)|_x \cdot (z - x)$$

In other words, β must satisfy

$$f(z) - f(x) - \nabla f|_x \cdot (z - x) \geq -\beta (g(z) - g(x) - \nabla g|_x \cdot (z - x))$$

Since f, g are convex, we know that this implies that

$$\beta \geq -\frac{f(z) - f(x) - \nabla f|_x \cdot (z - x)}{(g(z) - g(x) - \nabla g|_x \cdot (z - x))}$$

If the denominator is zero, we define the fraction to be $-\infty$. Then the threshold β^* can be computed as

$$\sup_{z \in A, x \in A} -\frac{f(z) - f(x) - \nabla f|_x \cdot (z - x)}{(g(z) - g(x) - \nabla g|_x \cdot (z - x))}$$

If the inequality

$$y_j \geq y_i + g_i^\top (x_j - x_i) \quad \text{for all } i, j$$

is satisfied, then one can explicitly construct a convex function $f(x)$ that interpolates the data.

Each pair (x_i, y_i) along with its subgradient g_i defines an affine function:

$$f_i(x) = y_i + g_i^\top (x - x_i).$$

If the inequality condition holds, we can take the maximum over these affine functions to define a convex interpolant:

$$f(x) = \max_{i=1, \dots, n} \{y_i + g_i^\top (x - x_i)\}.$$

This $f(x)$ is convex because the pointwise maximum of affine functions is convex. Moreover, it satisfies $f(x_i) = y_i$, hence it interpolates the given data.

2 Local Convexity

Let $f : A \rightarrow \mathbb{R}$, $A \subset \mathbb{R}^d$. We say that f is locally convex at x if there is a neighborhood U of x such that for all $z \in U$,

$$f(z) - f(x) \geq \nabla f|_x \cdot (z - x)$$

Let $f, g : A \rightarrow \mathbb{R}$ be convex. We want to find the minimum β such that $f + \beta g$ is convex at A . For each x , we can find β_x such that $f + \beta g$ is locally convex at x for all $\beta \geq \beta_x$. Then, we can set the threshold $\beta^* = \sup_x \beta_x$. β_x can be computed as follows:

$$\beta_x = \sup_{z \in U} - \frac{f(z) - f(x) - \nabla f|_x \cdot (z - x)}{(g(z) - g(x) - \nabla g|_x \cdot (z - x))}$$

where U is a neighborhood around x .

- For one pair, $\beta \geq 1$
- For another, maybe $\beta \geq -2$
- For another, $\beta \geq 5$

If you want a single β that works **for all pairs**, you need to pick the **largest** of these lower bounds.

That's why:

$$\beta^* := \sup_{x, z \in A} \left(\frac{-\text{numerator}}{\text{denominator}} \right)$$

This gives the **tightest possible threshold** — the smallest β that satisfies *all* those inequalities.

3 Numerical Simulation

This algorithm has the following parameters: r, N, M and the set A . First, we sample $x_1, \dots, x_N$ uniformly random from A . This can be easily achieved if A is a rectangular box. Then, we sample M points from the $d - 1$ sphere of radius r . This can be done by sampling $X \sim N(0, 1)^d$. Then, $X/\|X\|$ will be uniformly distributed in the sphere (This idea is suggested by Sahith). For each $i \in [N], j \in [M]$, we define $z_{i,j} = x_i + X_j/\|X_j\|$. Then, we can compute β as

$$\beta = \max_{i,j} - \frac{f(z_{i,j}) - f(x_i) - \nabla f|_{x_i} \cdot (z_{i,j} - x_i)}{(g(z_{i,j}) - g(x_i) - \nabla g|_{x_i} \cdot (z_{i,j} - x_i))}$$

Parameters:

- r : radius of a sphere used to perturb each point.
- N : number of base points $x_1, \dots, x_N$ sampled from the domain A .
- M : number of perturbation directions per point.
- A : domain (e.g., a box in $\mathbb{R}^d$).

Sampling Procedure:

Base Points:

- Sample $x_1, \dots, x_N \in A$ uniformly at random.
- Each x_i is a point in the domain.

Perturbation Directions:

- Sample M random vectors $X_j \sim \mathcal{N}(0, I_d)$, i.e., from a standard normal in $\mathbb{R}^d$.
- Normalize them to unit length: $\frac{X_j}{\|X_j\|}$ lies on the *unit sphere* S^{d-1} .
- Scale to get points on the **radius** r sphere: $x_i + r \cdot \frac{X_j}{\|X_j\|}$.
- Denote this perturbed point as:

$$z_{i,j} := x_i + \frac{X_j}{\|X_j\|} \cdot r.$$

Objective:

For each pair (i, j) , compute the expression:

$$\frac{f(z_{i,j}) - f(x_i) - \nabla f|_{x_i} \cdot (z_{i,j} - x_i)}{g(z_{i,j}) - g(x_i) - \nabla g|_{x_i} \cdot (z_{i,j} - x_i)}$$

Clarifying the Dimensions and Geometry

Sampling from $\mathbb{R}^d$:

Each random vector $X_j \sim \mathcal{N}(0, I_d)$ is a point in:

$$X_j \in \mathbb{R}^d$$

So the set of all such vectors spans $\mathbb{R}^d$, a d -dimensional Euclidean space.

Unit Sphere $S^{d-1} \subset \mathbb{R}^d$:

The **unit sphere** in $\mathbb{R}^d$ is defined as:

$$S^{d-1} = \{v \in \mathbb{R}^d \mid \|v\| = 1\}$$

- Although it's embedded in $\mathbb{R}^d$, it is a $(d-1)$ -dimensional **manifold**.
- So when we normalize X_j to:

$$s_j := \frac{X_j}{\|X_j\|} \in S^{d-1}$$

we are taking a vector that *lives* in $\mathbb{R}^d$, but now lies *on* the surface of the sphere — which has only $d-1$ degrees of freedom (since one degree is used up by the constraint $\|s_j\| = 1$).

Scaled Sphere of Radius r :

Now scale this unit vector to lie on the sphere of radius r :

$$r \cdot s_j = r \cdot \frac{X_j}{\|X_j\|} \in \mathbb{R}^d$$

This is still a vector in $\mathbb{R}^d$, but it lies on the surface of a sphere of radius r .

Perturbation Point:

To perturb a base point $x_i \in \mathbb{R}^d$, we define:

$$z_{i,j} := x_i + r \cdot \frac{X_j}{\|X_j\|} \in \mathbb{R}^d$$

So:

- $x_i \in \mathbb{R}^d$
- $z_{i,j} \in \mathbb{R}^d$
- $z_{i,j}$ lies on the sphere of radius r centered at x_i

4 Potential Improvement

I think if we let the variable x distribute regularly evenly (equidistribution) as opposed to uniformly random, we might get a more accurate result or need less datapoint. We can achieve this if we let x distributed on the lattice on d -dimensional space. However, I do not know how to make X distributed regularly even on the $n - 1$ -dimensional sphere. There is an article for equidistribution on S^1 (Sphere on $\mathbb{R}^2$). Also, I have implemented code on python, but I think the code runs very slowly. We might need to see how to improve the speed of the program. By testing, increasing the dimension (until 10-dim) will not increase the running time of the program for too much.